PQMigrate: Evidence-Driven and Abstention-Aware Planning for Post-Quantum Cryptographic Migration

Deepak Uplaonkar, Ratnesh Patnaik, Saravanan Kumar,
Vaibhavi Shah, and Prince Ostwal

Affiliations and corresponding-author email pending

Abstract— Post-quantum cryptographic migration requires identifying how an algorithm is used, not merely detecting its name. An RSA reference may denote a signature, encryption, key transport, or an unused dependency, each requiring different migration decisions. This paper presents PQMigrate, a bounded static-analysis and migration-planning prototype that associates source evidence with operation roles, versioned advisory rules, explicit blockers, and abstention. Its architecture supports Python and Go analysis and exports traceable findings through JSON, SARIF, and a cryptographic bill of materials. We evaluate the Python RSA resolver using a completed, single-author, 72-case synthetic pilot, comprising 52 operation cases, 16 negative controls, and four separately accounted malformed-input cases. On the 68 parseable cases, the full resolver obtains operation precision 1.000, recall 0.769, and F1 0.870. It assigns the correct role to all 40 answered gold operations while leaving 12 unsupported operations unanswered. A method-name AST baseline obtains higher detection F1 (0.880) but lower conditional role accuracy (0.727). These descriptive results illustrate why migration assessment should report both coverage and role correctness. They do not establish generalization, statistical superiority, safe recommendation rates, or successful post-quantum transformation. We describe the implemented assurance workflow, its reproducibility evidence, and a separately marked plan for independent, repository-separated evaluation. The contribution is an inspectable planning prototype and a bounded pilot study with explicit evidence limits.

Index Terms: post-quantum cryptography, cryptographic migration, static analysis, abstention, cryptographic inventory, reproducibility.

I. INTRODUCTION

Migration to post-quantum cryptography (PQC) is a software-engineering problem as well as an algorithm-selection problem. NIST standardizes ML-KEM for key encapsulation and ML-DSA for digital signatures in FIPS 203 and FIPS 204, respectively [1], [2]. These standards define different cryptographic functions. Finding the token `RSA` is therefore insufficient to select a replacement: a signature interface and an RSA encryption interface do not have interchangeable requirements. Migration and cryptographic-agility research also emphasizes dependencies, deployed interfaces, and coordinated change [3], [4].

Consider a program that generates an RSA key and later signs an application message. A planner can identify a signature role and suggest evaluating an ML-DSA-based design while retaining requirements for provider support, serialization, verification peers, and tests. If RSA instead encrypts a newly generated session secret, planning concerns key transport and a possible KEM-based redesign. A KEM establishes a shared secret; it is not a general-purpose replacement for an `encrypt(message)` API. An import with no linked use supports an inventory lead but no resolved migration role.

PQMigrate makes this distinction explicit. It produces evidence-linked observations, attempts bounded role inference, applies a versioned knowledge base, and exposes conditional advice or abstention. Missing evidence remains a blocker. Advice does not authorize mutation. The research question is whether this design makes detection coverage, role correctness, and unresolved context measurable and reviewable.

The contributions are threefold. First, we describe an implemented evidence-to-decision architecture with typed findings, versioned rules, deterministic decision traces, and explicit refusal conditions. Second, we report a reproducible synthetic pilot comparing lexical lookup, method-name AST matching, and the full Python RSA resolver, together with two output-masking sensitivity checks. Third, we document engineering validation and specify an independent evaluation protocol for future work. We make no first-of-kind claim, no claim of superiority over external analyzers, and no claim of validated automatic PQC transformation.

II. BACKGROUND AND RELATED WORK

A. Cryptographic discovery and migration planning

NIST's migration project distinguishes cryptographic discovery from interoperability and performance testing [5]. Cryptoscope combines cryptographic knowledge with parsing and data/control-flow analysis to identify cryptographic usage [6]. This is close prior work: operation-aware discovery already exists, so semantic analysis alone is not a sufficient novelty claim. PQMigrate emphasizes the inspectable connection between bounded source evidence, unresolved planning obligations, and abstention. We do not execute Cryptoscope or compare its published numbers with this pilot's numbers.

Hasan et al. organize migration around inventories and cryptographic dependencies, including graph-based case studies [4]. PQMigrate addresses a narrower source-level task: explaining why an observation supports an advisory family or remains unresolved. It does not replace an enterprise dependency inventory or estimate organizational migration cost.

B. API analysis and evidence interchange

CogniCrypt assists developers with cryptographic code generation and static checks of API use [7]. It demonstrates the importance of task context and integration checks; its task and evaluation differ from the Python RSA role-resolution pilot here. It is related work rather than an executed baseline.

CycloneDX CBOM represents cryptographic assets and their relationships [8], while SARIF provides an interchange format for static-analysis results [9]. PQMigrate uses these formats to carry findings and provenance. Schema validity establishes structural conformance, not the truth or completeness of a finding, downstream ingestion success, or system security.

C. Abstention and coverage

Selective prediction studies the trade-off between answering and rejecting uncertain inputs [10]. PQMigrate adopts the distinction between coverage and correctness among answers. Its rejection policy is rule-based, with categorical evidence labels rather than calibrated probabilities. It does not inherit statistical risk guarantees from selective-classification research. A refusal to guess can still be a missed operation, and reporting both outcomes is necessary.

III. PROBLEM DEFINITION AND RESEARCH QUESTIONS

A. Analysis unit and outputs

The system accepts a source tree, a detection registry, and a versioned migration knowledge base. A finding records a primitive, source location, operation, role, protocol context, confidence category, and evidence. A plan adds the selected rule, advisory target family, unresolved requirements, blockers, and decision trace.

The pilot uses a more restricted unit: one Python snippet mapped to one resolver result. It is not a benchmark of every operation within a multi-operation file. The pilot roles are signature, key transport, encryption, and unknown; operations are sign, verify, encrypt, decrypt, and unknown. Protocol labels are JWT and unknown. Because the evaluator defines a predicted operation as a non-unknown role, operation detection and role resolution are coupled in this pilot.

B. Research questions

1. **RQ1:** How do lexical lookup, method-name AST matching, and bounded identity/context inference differ in operation detection on the pilot?

2. **RQ2:** What role correctness is obtained at the observed answer coverage, and which cases remain unresolved?

3. **RQ3:** How sensitive are role and protocol metrics to masking session-secret and protocol output information?

4. **RQ4:** What reproducibility and assurance properties are demonstrated by the implemented artifact workflow?

Whether abstention reduces unsafe migration recommendations on independently labeled real code remains open. No completed experiment here directly estimates that rate.

IV. SYSTEM ARCHITECTURE AND REQUIREMENTS

A. Evidence pipeline

The implementation follows six stages: source discovery; parsing and bounded inference; typed CryptoIR construction; knowledge-base matching; advisory planning and tracing; and export or presentation. Python analysis uses the standard AST. Go inventory discovery is supplemented by Tree-sitter-based bounded semantic analysis. Implementation support is distinct from measured coverage: the quantitative study contains Python RSA snippets only.

Table I. Pipeline modules and evidence boundaries. The dashboard presents artifacts rather than adding semantic inference.

Module	Input and output	Evidence boundary
Discovery and parsing	Files to leads and syntax	No target execution during scanning
Role resolver	Imports, assignments, calls to role/context	Bounded scope and patterns
CryptoIR	Findings to typed records and locations	Unknown fields retained
Knowledge and planner	Findings to conditional advice	Versioned rules and blockers
Trace and export	Decisions to JSON, SARIF, CBOM	Provenance and schema checks
Assurance and UI	Evidence to checks and views	Local validation; apply disabled

B. Functional and assurance requirements

The system must distinguish inventory leads from resolved uses, preserve locations and evidence, represent unknown or conflicting roles, identify matching rules and versions, and retain blockers in exported decisions. The planner must not turn an advisory target into permission to modify code. Categorical confidence and priority are not probabilities.

The knowledge loader rejects duplicate identifiers, missing metadata, unsupported taxonomy values, and mutation-enabling rule settings. Current advisory rules require manual intervention. Decision traces record detection, inference, rule matching, blocker assessment, and outcome. Their identifiers hash canonical semantic inputs while excluding scan identifiers and timestamps. Equivalent pinned semantic inputs therefore preserve trace identity; timestamped artifacts need not be byte-identical across runs.

C. Trust model

Repositories are untrusted inputs. The analyzer parses source rather than importing or executing the target program. This reduces an execution pathway but does not establish parser hardening, resource isolation, or immunity to hostile inputs. Operators must review evidence and migration constraints. Cryptographic security follows neither from a finding nor from a successful test gate.

V. METHODOLOGY, MATHEMATICAL MODEL, AND ALGORITHM ANALYSIS

A. Evidence and role inference

Let an observation be x, its evidence be E(x), and its inferred role be ρ(x). Evidence includes resolved import names, source spans, supported key-generation assignments, supported random-secret assignments, and linked calls in a module or function scope. Let S, T, and C denote signature, key transport, and encryption. A compact model is

$$\rho(x) = \begin{cases} S & \text{if linked signature use} \\ T & \text{if encryption of a tracked secret} \\ C & \text{if encryption, purpose unresolved} \\ \perp & \text{if no supported use or conflicting roles} \end{cases} \quad (1)$$

The encryption case also includes linked decryption of unresolved purpose. Here ⊥ means unknown. These cases summarize matching rules, not proven runtime behavior. A locally generated random value is the implementation's proxy for key-transport purpose; it does not independently prove later use as a session key.

The Python resolver collects imports and same-scope assignments, records supported RSA key generation and random-secret producers, and examines calls in source-line order. JWT is recognized when `jwt.encode` consumes a tracked RSA key and a literal algorithm string satisfying the implemented `RS` prefix check. This is not a complete JWT algorithm registry or protocol validation. Sign/verify methods on tracked roots support a signature role. Encryption of a tracked random value supports key transport; other linked encryption/decryption receives the broader encryption role.

Results are aggregated across the snippet. Multiple observed roles cause an unknown result with conflict evidence; otherwise the earliest matching result is selected. This is not a complete per-call inventory. Reassignment, aliasing, arbitrary wrappers, branch feasibility, cross-function flow, and dynamically selected providers are not established. Identity tracking is a syntactic approximation rather than a sound points-to analysis.

B. Planning and abstention

Let K_v be the knowledge base at version v. The planning function is

$$\pi(x, K_v) = \begin{cases} \text{advise}(t, B) & \text{if a compatible rule matches} \\ \text{abstain}(B) & \text{otherwise} \end{cases} \quad (2)$$

where t is a target family and B contains blockers or unresolved requirements. Advice is conditional: a signature role may support ML-DSA assessment, while key transport supports assessment of a KEM/data-encryption construction. Neither establishes provider availability, peer compatibility, key storage behavior, or application semantics.

Automatic application is governed by an assurance predicate

$$G(x) = R(x) \wedge C(x) \wedge I(x) \wedge T(x) \wedge A(x) \quad (3)$$

whose terms denote a known role, supported construction, interoperability evidence, required tests, and operator authorization. A false or unknown prerequisite prevents eligibility. This is a policy contract, not a cryptographic security theorem. Current RSA plans retain `patch_available=false`.

C. Bounded algorithm

```
Input: source snippet and versioned knowledge base
1. Parse syntax; retain parse failure evidence.
2. Collect imports and local assignment observations.
3. Record supported RSA keys and random values.
4. Examine linked calls in source-line order by scope.
5. Resolve a unique role or retain unknown/conflict.
6. Match an advisory rule and record its version.
7. Emit advice or abstention with trace and blockers.
8. Refuse application with unmet prerequisites.
```

Algorithm 1. Evidence-to-plan procedure. The pilot evaluates steps 1-5; the planner adds steps 6-8.

D. Complexity and implementation cost

Let N be the Python syntax-node count and n_s the collected nodes in scope s. Parsing and traversal are ordinarily linear in input size, but the implementation sorts each scope's nodes twice by source line. With bounded identifier lengths and average constant-time dictionary access, one resolver invocation therefore costs $O(N+\Sigma_s n_s \log n_s)$, bounded by $O(N \log N)$ when nodes are counted a bounded number of times across scopes. Memory is O(N+F), where F counts accumulated results. Very long names and malformed-input parser behavior require separate consideration.

Reanalyzing a file for L inventory leads can multiply total work by L; this per-invocation bound is not a proven whole-scanner bound. Go inventory matching also depends on pattern count and source bytes. No scaling experiment or production throughput claim is reported. These bounds motivate profiling and caching studies.

VI. COMPLETED PILOT: EXPERIMENTAL DESIGN

A. Corpus and labels

PQMigrateBench v0.1 contains 72 single-author synthetic Python RSA cases (Table II). Detection and role metrics score 68 parseable cases: 52 labeled operations and 16 negative controls. Four designated malformed inputs are retained separately. Unsupported but syntactically valid operations remain in the 52-operation denominator; they are not removed because the resolver cannot handle them.

Table II. Completed pilot composition. Outcomes describe the resolver, not migration recommendations.

Category	Cases	Full resolver outcome
Direct signature	12	12 correct roles
JWT signature	8	8 correct roles and contexts
Key transport	12	12 correct roles
Asymmetric encryption	8	8 correct roles
Unsupported valid operations	12	12 unanswered, counted as misses
Negative controls	16	16 negative predictions
Malformed inputs	4	Separately accounted
Total	72	68 scored for detection/role

Nominal train, development, and test assignments contain 44, 14, and 14 cases. They are synthetic partitions, not

independently collected repository holdouts. The resolver is not a learned model. Naming a partition "test" does not establish independence from author decisions or related scenario leakage. Labels have not undergone independent annotation or adjudication, and inter-rater agreement is unavailable.

B. Executed baselines and sensitivity checks

The lexical baseline flags an operation when a lowercased snippet contains `rsa`; otherwise it returns unknown. It does not resolve imports and always leaves role and operation labels unknown. It is an intentionally weak inventory heuristic, not a representative implementation of every lexical analyzer.

The AST baseline matches attribute calls named sign, verify, encrypt, or decrypt without resolving receiver identity. It returns a result only when observed method-derived roles are unique. Sign/verify maps to signature; encrypt/decrypt maps to encryption. It does not distinguish transported key material or recognize the JWT wrapper.

The full configuration calls the bounded resolver in Section V. Two further configurations reuse its predictions: the session-secret mask rewrites key-transport roles as encryption; the protocol mask replaces protocol outputs with unknown. Although the artifact labels these "ablations," neither reruns inference with an internal stage disabled. They measure sensitivity to output information rather than causally isolating analysis components.

All configurations use identical case IDs, labels, assignments, and accounting classes. Malformed cases are excluded from detection/role denominators for every configuration, including the non-parsing lexical baseline. The harness does not implement uniform per-case timeout/crash measurement; no comparative robustness result is claimed. Regex-call and external-tool baselines are planned, not executed.

C. Metrics and denominators

On the 68 scored cases, detection uses true positives, false positives, false negatives, and true negatives:

$$P = TP/(TP + FP),\ R = TP/(TP + FN)$$
$$F_1 = 2PR/(P + R) \tag{4}$$

Let O be the 52 gold-operation cases, $A \subseteq O$ those predicted operation-positive, and $C \subseteq A$ those with the correct role. We report

$$\text{Coverage}_O = |A|/|O|,\ \text{Acc}_A = |C|/|A|$$
$$\text{Acc}_O = |C|/|O| \tag{5}$$

These are gold-operation-conditioned measures. Conditional role accuracy excludes false-positive operations on negative controls and must be read alongside precision. The lexical baseline can be "answered" under this detection definition despite returning an unknown role. A future common harness will explicitly distinguish detection from a usable role answer.

The operation non-answer rate is 1-Coverage_O; it is not the planner's abstention rate. Exact operation/role accuracy uses all 68 scored cases, requiring correct presence and, for gold operations, correct role and operation labels. Protocol accuracy uses the eight cases with known gold context. Error count is the number of cases with at least one scored mismatch, not the number of unsafe recommendations.

VII. PILOT RESULTS AND ERROR ANALYSIS

A. Detection and role outcomes

Tables III and IV report completed aggregate results. Values are rounded to three decimals; artifacts retain six decimals and raw predictions.

Table III. Detection on 68 parseable pilot cases.

Configuration	TP/FP/FN/TN	Precision	Recall	F1
Lexical RSA lookup	40/16/12/0	0.714	0.769	0.741
AST method matching	44/4/8/12	0.917	0.846	0.880
Session-secret output mask	40/0/12/16	1.000	0.769	0.870
Protocol output mask	40/0/12/16	1.000	0.769	0.870
Full resolver	40/0/12/16	1.000	0.769	0.870

Table IV. Role metrics use 52 gold operations; A denotes answered gold operations. Error counts use the 68-case set and include labeled protocol mismatches.

Configuration	Coverage on O	Role accuracy on A	Role accuracy on O	Error cases
Lexical RSA lookup	0.769	0.000	0.000	68
AST method matching	0.846	0.727	0.615	24
Session-secret output mask	0.769	0.700	0.538	24
Protocol output mask	0.769	1.000	0.769	20
Full resolver	0.769	1.000	0.769	12

The AST baseline detects four more gold operations than the full resolver but also produces four false positives. Its 0.880 F1 exceeds the full resolver's 0.870. However, it returns 32 correct roles among 44 answered gold operations, versus 40 among 40 for the full resolver. End-to-end role accuracy is consequently 40/52 = 0.769 for the full resolver and 32/52 = 0.615 for AST matching. These are descriptive differences, not statistically established effects.

All 12 full-resolver errors belong to the unsupported-valid-operation category. Non-answer rate is 12/52 = 23.1% among gold operations. Across 68 scored cases, 28 predictions are operation-negative: 12 missed operations and 16 correctly rejected negative controls. Calling all 28 "abstentions on operations" would conflate outcomes. Exact operation/role accuracy is 56/68 = 82.4%, combining 40 correctly resolved operations and 16 correctly rejected negatives.

B. Sensitivity to output information

Masking session-secret information changes 12 correct key-transport roles to encryption. Detection counts remain unchanged while end-to-end role accuracy falls from 40/52 to 28/52. Detection metrics therefore cannot reveal the role distinction erased by the mask. This does not measure how a separately implemented analyzer without secret tracking would behave.

The protocol mask preserves operation and role predictions but changes eight correct JWT contexts to unknown. Context accuracy falls from 8/8 to 0/8 and error cases rise from 12 to 20. This demonstrates the accounting value of reporting context separately, with no evidence about TLS, SSH, certificates, or other contexts.

C. Nominal split results

Table V. Full-resolver results by synthetic assignment. Each partition has zero false positives and conditional role accuracy 1.000. These are not repository-separated generalization estimates.

Synthetic split	Total / scored	Gold ops.	TP/FN	F1	Coverage on O
Train	44/42	32	24/8	0.857	0.750
Development	14/13	10	8/2	0.889	0.800
Test	14/13	10	8/2	0.889	0.800

D. Failures and unmeasured outcomes

Malformed inputs constitute 4/72 = 5.6% of the corpus. This is a designed-case fraction, not an estimated parser-failure rate on natural code. Diagnostics remain outside the principal score denominator. The current error taxonomy distinguishes presence, role, operation-label, and protocol mismatches; it does not establish every unsupported case's root cause independently.

No bootstrap intervals, independent-annotation agreement, deployment success rates, or false-recommendation rates have been completed. Forty correct roles among 40 answers do not establish zero future error, and zero detection false positives do not establish safe migration advice. A naive case-level interval would not remedy the authored corpus and correlated scenarios.

VIII. IMPLEMENTATION ASSURANCE AND REPRODUCIBILITY

A. Completed engineering evidence

The D0-D20 integration gate passed twice from a clean detached checkout of the same implementation commit. Preserved logs record 97 Python tests, 23 legacy scanner checks, five Maven tests, and a successful React/Vite build, together with later milestone scripts. These are software checks, not additional independently labeled research cases.

The preview/verification workflow demonstrates a narrow MD5-to-SHA-256 expression transformation. It binds a preview to a source hash, materializes a disposable copy, checks Python syntax and expected AST structure, and verifies that original source remains unchanged. This does not establish application-level semantic equivalence. The temporary filesystem tree is not a container, virtual machine, or full security sandbox.

The two-party laboratory evaluates a controlled digest-negotiation contract with agreement, fallback, no-common-mode, and tampered-digest scenarios. It does not implement ML-KEM, ML-DSA, or a standardized PQC hybrid protocol. These checks demonstrate the assurance workflow, not successful PQC migration.

Deployment evidence identifies a locally validated WSL CLI, loopback dashboard, local static-site preview, backend tests, and frontend build. There is no public hosting or production-deployment claim. Release tagging and faculty/security review remain pending. Dependency-audit findings require triage before release readiness.

B. Source and artifact identity

The source repository is available at the project URL [13]. The evaluated implementation commit is

`bcfb5e40fbc8eb74e1c919114f54031d03ab44e9`

The SHA-256 of `benchmarks/pqmigratebench_v0_1.jsonl` is

`293fee315c4fbcb95f0d071615ce54a7ac6c9b8b89783a832832c15e3697603b`

The full gate is `bash scripts/validate_d0_d20.sh`; the focused baseline gate is `bash scripts/run_d18_baselines.sh`. The D20 manifest records the source commit and milestone-artifact hashes. Complete logs, artifacts, and checksums were preserved separately under `pqmigrate-release-evidence/bcfb5e4/run1/` and `run2/` in the local environment. This avoids changing the validated source commit to hold regenerated evidence. Local preservation is verified; an immutable public evidence deposit has not been established.

Reproduction requires the documented Python dependencies and Tree-sitter grammar, Java/Maven, and Node/npm dependencies. Historical runs reused the configured project environment. Clean source checkout is supported; a hermetic build from a new machine is not established. Later documentation revisions do not change the identity of the evaluated implementation.

IX. DISCUSSION

The pilot illustrates that higher detection F1 can coexist with fewer correct migration-relevant roles. Distinguishing transported key material from generic encryption matters even when both use an encryption call. Preserving unresolved context is useful for review, although it does not itself establish safety.

Abstention has a cost: 12 genuine operations remain unresolved and require investigation. Favorable conditional accuracy is obtained on a deliberately bounded subset. Expanding that subset could improve coverage while introducing wrong answers. Measuring this trade-off on independent code is the next empirical task.

The workflow also distinguishes syntax checking, schema validation, source preservation, peer compatibility, and cryptographic correctness. Passing one obligation does not discharge the others. This can organize migration reviews, but reviewer effort and decision quality have not been evaluated in a user study.

X. LIMITATIONS AND THREATS TO VALIDITY

Construct validity. Snippet-level summaries couple detection to a known-role result. They do not measure complete per-call extraction, planner abstention, or recommendation safety. Random-source tracking is a proxy for key-transport purpose; evidence strength is not calibrated confidence.

Internal validity. One author constructed the corpus and labels, and development had access to scenario families. The lexical baseline is weak; no external analyzer was executed. Output masking is not internal component ablation. Bounded-analysis limitations include reassignment, aliasing, wrappers, cross-scope flow, and mixed-role aggregation. Tests do not establish soundness.

External validity. Quantitative evidence is synthetic Python RSA only. It cannot establish Go accuracy, real-repository performance, other primitive/protocol coverage, or production dependency behavior. Existing repository scans lack the independent gold labels needed for accuracy claims. The nominal test partition is not a repository holdout.

Statistical conclusion validity. Confidence intervals, significance tests, independent agreement results, and population sampling are absent. Scenario correlation and purposive construction prevent treating snippets as independent natural-code draws. Perfect conditional accuracy on 40 selected answers must not be advertised as general reliability.

Security and deployment validity. Advisory planning does not prove confidentiality, unforgeability, or secure composition. Digest-contract tests provide no PQC interoperability evidence. Application remains disabled, public deployment is unverified, and faculty/security approval and release tagging are incomplete. Concrete integrations must check current standards and implementation errata.

XI. PLANNED EVALUATION AND FUTURE WORK

A. Independent corpus and annotation: not yet completed

The next evaluation will target 200-300 reviewed cases, with a working target of 240 across multiple repositories and balanced between Python and Go. Categories will include signatures, key transport, encryption, key establishment/protocols, negative controls, unsupported valid operations, ambiguous evidence, and malformed inputs. Inclusion criteria, revisions, licenses, spans, discovery methods, and sampling decisions will be recorded. Multiple candidate-discovery methods will reduce selection toward existing strengths.

Two human annotators will independently label presence, role, context, support status, and advisory correctness, blinded to tool predictions. Calibration cases will be excluded from final testing. Original labels, disagreements, rationales, and adjudicated labels will be retained. Cohen's kappa [11] will be reported per applicable nominal field with raw agreement and label prevalence. For observed agreement p_o and chance agreement p_e,

$$\kappa = (p_o - p_e)/(1 - p_e) \tag{6}$$

Repository families, forks, and duplicate snippets will remain within a partition. An approximately 60/20/20 allocation will be fixed by repository, with the test set unavailable for rule tuning. Final sizes and allocations will be reported as observed rather than assumed to match targets.

B. Baselines and statistics: not yet completed

A common harness will compare primitive/import lookup, regex-call matching, AST matching, the full pipeline, and a pinned Semgrep Community Edition configuration with published rules [12]. The latter is an external analysis-engine baseline, not a claim that custom Semgrep rules form a complete migration product. All systems will use identical cases, gold labels, output definitions, limits, timeout policies, and failure accounting. Parse failures, unsupported cases, crashes, and timeouts will remain in an explicit ledger.

Analysis will report detection, role accuracy, useful answer coverage, correctness among answers, end-to-end accuracy, planner abstention, false recommendations, parser failures, and runtime by language, category, and held-out split. Unsafe recommendations will require independently adjudicated advisory labels; detection false positives will not substitute for that outcome. Rates will be reported both per recommendation issued and per eligible evaluation case, with explicit denominators.

Planned 95% intervals use 10,000 repository-cluster bootstrap replicates, preserving within-repository dependence and pairing systems on the same resampled repositories. Undefined metrics and small-cluster limitations will be disclosed. Current tables contain none of these future estimates or intervals.

C. Transformations and separate research tracks

Future PQMigrate work will evaluate a bounded RSA signature interface integrated with an ML-DSA provider and a bounded key-establishment fixture using a specified ML-KEM or standardized hybrid construction. Provider versions, test vectors, formats, failure handling, cross-party tests, and performance measurements must precede transformation claims. No such result is part of this manuscript.

The eight-state superposition/public-key investigation is a separate proposed study. It contributes no construction, attack measurement, security proof, or result to this paper. Any future connection requires its own threat model, experiment, and security argument.

XII. CONCLUSION

PQMigrate implements an evidence-linked planner with bounded role inference, versioned advice, and explicit abstention. Its completed 72-case synthetic pilot demonstrates the distinction between operation detection and role correctness: the full Python RSA resolver answers 40 of 52 gold operations with correct roles and leaves 12 unsupported operations unresolved. AST matching has slightly higher detection F1 but lower conditional role accuracy. These observations support reporting coverage and semantic correctness together; they do not establish general superiority or safe automatic migration. The artifact supplies a reproducible starting point for independent real-code evaluation and separately validated PQC integrations.

REFERENCES

[1] National Institute of Standards and Technology, *Module-Lattice-Based Key-Encapsulation Mechanism Standard*, FIPS 203, Aug. 2024, doi: 10.6028/NIST.FIPS.203. https://csrc.nist.gov/pubs/fips/203/final

[2] National Institute of Standards and Technology, *Module-Lattice-Based Digital Signature Standard*, FIPS 204, Aug. 2024, doi: 10.6028/NIST.FIPS.204. https://csrc.nist.gov/pubs/fips/204/final

[3] D. Ott, C. Peikert, and other workshop participants, "Identifying Research Challenges in Post Quantum Cryptography Migration and Cryptographic Agility," Computing Community Consortium workshop report, arXiv:1909.07353, 2019. https://arxiv.org/abs/1909.07353

[4] K. F. Hasan et al., "A Framework for Migrating to Post-Quantum Cryptography: Security Dependency Analysis and Case Studies," *IEEE Access*, 2024, doi: 10.1109/ACCESS.2024.3360412; author manuscript arXiv:2307.06520v2. https://arxiv.org/abs/2307.06520

[5] NIST National Cybersecurity Center of Excellence, "Migration to Post-Quantum Cryptography," project resource, accessed Oct. 4, 2026. https://www.nccoe.nist.gov/applied-cryptography/migration-to-pqc

[6] M. Moffie et al., "Cryptoscope: Analyzing cryptographic usages in modern software," arXiv:2503.19531, 2025, doi: 10.48550/arXiv.2503.19531. https://arxiv.org/abs/2503.19531

[7] S. Kruger et al., "CogniCrypt: Supporting Developers in using Cryptography," author manuscript, 2017. https://www.eclipse.org/cognicrypt/resources/publications/knr%2B17cognicrypt.pdf

[8] OWASP Foundation, "Cryptography Bill of Materials (CBOM)," CycloneDX documentation, accessed Oct. 4, 2026. https://cyclonedx.org/capabilities/cbom/

[9] OASIS, *Static Analysis Results Interchange Format (SARIF) Version 2.1.0 Errata 01*, OASIS Standard. https://docs.oasis-open.org/sarif/sarif/v2.1.0/errata01/os/sarif-v2.1.0-errata01-os.html

[10] Y. Geifman and R. El-Yaniv, "Selective Classification for Deep Neural Networks," arXiv:1705.08500, 2017, doi: 10.48550/arXiv.1705.08500. https://arxiv.org/abs/1705.08500

[11] J. Cohen, "A Coefficient of Agreement for Nominal Scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37-46, 1960, doi: 10.1177/001316446002000104. https://journals.sagepub.com/doi/abs/10.1177/001316446002000104

[12] Semgrep, "Run rules," documentation, accessed Oct. 4, 2026. https://docs.semgrep.dev/running-rules

[13] PQMigrate project, "PQMigrate source code and evaluation artifacts," Git repository, evaluated revision bcfb5e40fbc8eb74e1c919114f54031d03ab44e9, 2026. https://github.com/Ratneshp1122/PQMigrate/tree/bcfb5e40fbc8eb74e1c919114f54031d03ab44e9